axiateam
RDT company

Módulo 12

Testing con TypeScript

Curso: TypeScript

120 min

Objetivos de Aprendizaje

- Configurar **Jest** para TypeScript
- Escribir **tests unitarios** con tipos
- Crear **mocks tipados**
- Implementar **tests de integración**
- Medir **coverage** de código

1. Contexto

Testing en TypeScript

Por qué Testing con Tipos

TypeScript mejora el testing:

```
// Jest con tipos
test("should create task", () => {
  const task = createTask("Test task", "high");

  // TypeScript sabe el tipo de task
  expect(task.title).toBe("Test task");
  expect(task.priority).toBe("high");
});
```

El autocompletado y la verificación de tipos hacen que los tests sean más confiables.

2. Configuración

Jest con TypeScript

ts-jest permite ejecutar tests TypeScript con Jest.

```
npm install -D jest ts-jest @types/jest
```

```
// jest.config.ts
export default {
  preset: "ts-jest",
  testEnvironment: "node",
  roots: ["<rootDir>/src", "<rootDir>/tests"],
  moduleNameMapper: {
    "^@/(.*)$": "<rootDir>/src/$1"
  }
};
```

Lab: Configurar Testing

Objetivo: Configurar Jest para el proyecto.

1. Instala dependencias
2. Crea `jest.config.js`
3. Añade scripts a `package.json`
4. Verifica configuración

3. Tests Unitarios

Escribiendo Tests

Los tests unitarios prueban funciones de forma aislada.

```
// src/utils/calculator.ts
export function add(a: number, b: number): number {
  return a + b;
}

// tests/calculator.test.ts
import { add } from "../src/utils/calculator";

describe("Calculator", () => {
  test("adds two numbers", () => {
    expect(add(2, 3)).toBe(5);
  });

  test("handles negative numbers", () => {
    expect(add(-1, -2)).toBe(-3);
  });
});
```

Lab: Tests del Sistema de Tareas

Objetivo: Escribir tests unitarios para el servicio de tareas.

1. Crea test para `createTask`
2. Crea test para `getAll`
3. Crea test para filtrado

Código del Lab

```
// tests/taskService.test.ts
import { TaskService } from "../src/services/taskService";

describe("TaskService", () => {
  let service: TaskService;

  beforeEach(() => {
    service = new TaskService();
  });

  describe("createTask", () => {
    it("should create a task with given title", () => {
      const task = service.createTask("New task", "high");

      expect(task.title).toBe("New task");
      expect(task.priority).toBe("high");
      expect(task.status).toBe("pending");
    });
```

4. Mocks Tipados

Mocks con Tipo

Los mocks tipados facilitan testing de servicios dependientes.

```
// Mock de repositorio
const mockTaskRepository = {
  findAll: jest.fn(),
  findById: jest.fn(),
  save: jest.fn(),
  delete: jest.fn()
} as jest.Mocked<ITaskRepository>;

// Mock con implementación
const mockDate = new Date("2024-01-01");
jest.spyOn(global, "Date").mockImplementation(() => mockDate);
```

Lab: Mocks para el Sistema de Tareas

Objetivo: Crear mocks tipados para el servicio.

1. Crea mock para el repositorio
2. Mockea funciones externas
3. Verifica llamadas con tipos

5. SpyOn y toThrow

SpyOn

Permite espiar métodos específicos de un objeto sin reemplazar toda la implementación.

```
const consoleSpy = vi.spyOn(console, "log");

console.log("test");
expect(consoleSpy).toHaveBeenCalledWith("test");

consoleSpy.mockRestore();
```

toThrow

Verifica que una función lance un error.

```
function validateAge(age: number): void {
  if (age < 0) {
    throw new Error("Age cannot be negative");
  }
}

test("throws error for negative age", () => {
  expect(() => validateAge(-1)).toThrow("Age cannot be negative");
});
```

Lab: Tests de Error y Spies

Objetivo: Implementar tests con spies y verificación de errores.

1. Usa `vi.spyOn` para espiar llamadas
2. Usa `toThrow` para verificar errores

Resumen

Puntos Clave

- ts-jest permite ejecutar tests TypeScript
- jest.fn() crea funciones mock
- --coverage mide la cobertura de código

Ejercicio Integrador

Sistema de Biblioteca

Caso 2: Biblioteca

Objetivo: Escribir tests para el sistema de biblioteca.

1. Configura Jest para el proyecto
2. Escribe tests para servicios
3. Mide coverage

Trabajo en grupos de 2-4 personas.

Tips y Gotchas

✅ Tips

- Arrange-Act-Assert (AAA) estructura tests claros
- Tests unitarios: prueba una cosa, un assertion
- Mocks tipados con `vi.fn()` y `vi.mock()`

⚠️ Gotchas

- `any` en mocks defeatiendo el propósito - tipa correctamente
- Tests asíncronos necesitan `async/await`
- Coverage no garantiza tests correctos, solo cantidad

Referencias

Recursos Adicionales

- Jest Documentation
- Mock Functions

Gracias!

Módulo 12 Completado

Siguiente: Anexo 3A - Webpack + React